Predicting Hospital Readmission in Diabetes Patients: A Machine Learning Approach

Nathan Gin, Alexander Yu, Nhan Nguyen
CS273A Machine Learning, Fall 2025
University of California, Irvine

Abstract

Hospital readmission among diabetic patients is a costly and clinically significant outcome. Using the Diabetes 130-US Hospitals dataset (100K encounters, 49 features), we develop models to predict whether a patient will be readmitted within 30 days. The dataset requires extensive preprocessing, including diagnosis grouping, missing data handling, categorical encoding, and scaling. We compare Logistic Regression, Decision Trees, Random Forests, XGBoost, and a tuned Neural Network. Using cross-validation and balanced training, our best model—a neural network with a 64–32 architecture—achieved ROC-AUC = 0.82 and recall = 0.71. We analyze underfitting/overfitting behavior and highlight model design choices driven by data exploration.

1 Introduction

Hospital readmissions among diabetic patients incur significant healthcare costs and reflect underlying clinical instability. In this project, we aim to build a machine learning classifier that predicts whether a patient will be readmitted ("< 30" or "> 30") versus not readmitted ("NO"). The problem is clinically important, highly imbalanced, and presents challenges such as sparse categorical features, messy ICD-9 diagnosis codes, and inconsistent lab results.

This project meets the course requirements by:

- Exploring the dataset with visual and statistical analysis.
- Implementing techniques not fully covered in class (e.g., neural networks, XGBoost).
- Performing cross-validation and model selection.
- Performing overfitting/underfitting analysis with complexity curves.

2 Dataset Exploration

The dataset contains 100,000+ hospital encounters from 130 U.S. hospitals over 10 years. Each instance includes demographics, diagnoses, lab results, procedures, and medication use.

2.1 Missingness and Data Irregularities

Several variables contained extensive missingness:

- `weight`, `payer_code`, `medical_specialty`: $> 70\%$ missing.

- `A1Cresult` and `max_glu_serum` contained categories such as "None", "Norm", and "> 200" that required numeric encoding.

These insights motivated the removal or numerical mapping of certain features.

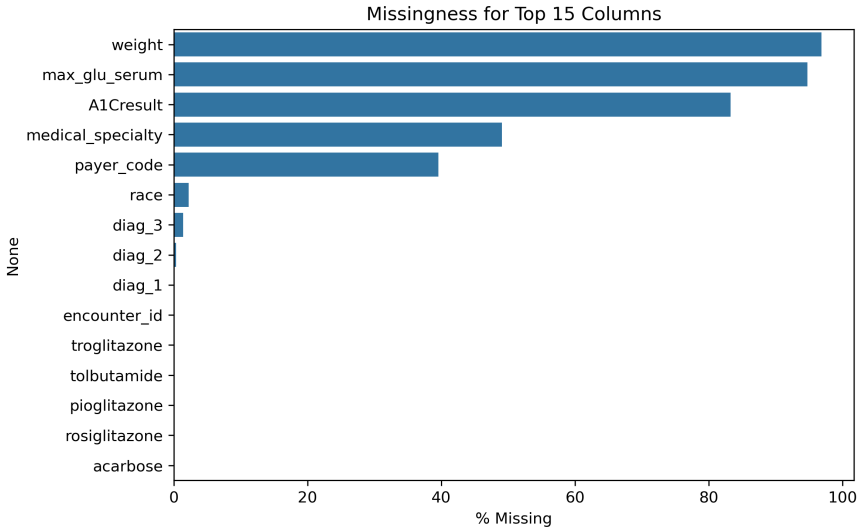

Figure 1: Missingness visualization across key columns.

2.2 Readmission Distribution

The dataset is severely imbalanced (Figure 2): only ∼11% of patients are readmitted, which motivates downsampling and class-weighting techniques.

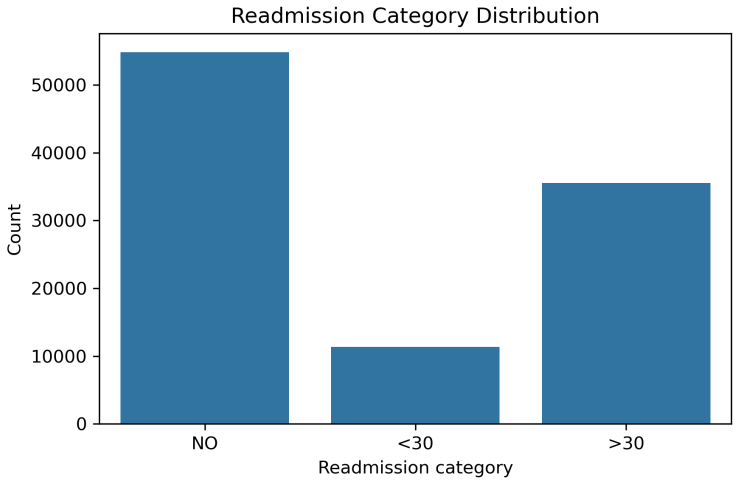

Figure 2: Readmission category distribution.

2.3 Exploratory Visualizations

We explored:

- Age distribution

- Number of inpatient/emergency visits
- Medication changes vs readmission
- Diagnosis code heterogeneity

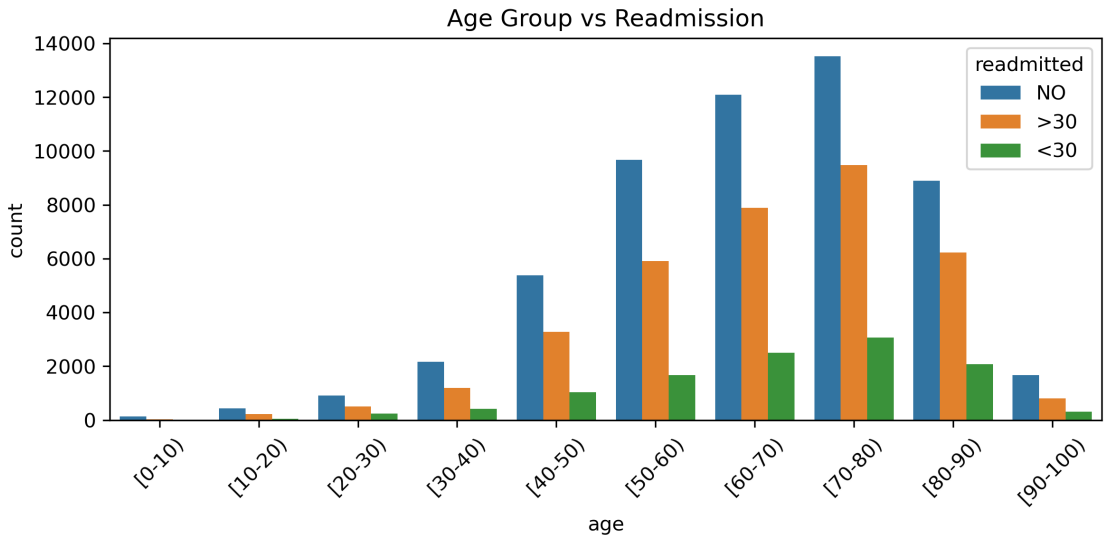

Figure 3: Age group vs readmission rate. Older groups show higher probability.

These visual patterns informed later feature engineering choices.

3 Feature Engineering

3.1 Diagnosis Grouping

Raw ICD-9 codes contain over 700 categories. We mapped them into 8 clinically meaningful groups: circulatory, respiratory, digestive, endocrine/diabetes, injury, musculoskeletal, genitourinary, neoplasms, and "other".

3.2 Medication Encoding

21 diabetes-related medications originally had 4 levels: `No`, `Up`, `Down`, `Steady`. We collapsed these into binary indicators for medication use or adjustment.

3.3 Categorical Encoding and Scaling

We one-hot encoded race and hospital admission/discharge IDs. Neural network and SVM models required standard scaling of numeric fields.

4 Modeling

4.1 Train–Test Split

We performed an 80/20 stratified split. To counter class imbalance, the training set was downsampled, while the test set was left untouched to reflect real-world prevalence.

4.2 Models Evaluated

- Logistic Regression
- Decision Tree (baseline + complexity analysis)
- Random Forest (grid-searched)
- XGBoost (boosting + `scale_pos_weight`)
- Neural Network (MLP) with hyperparameter tuning

4.3 Cross-Validation

We used 5-fold stratified cross-validation with ROC-AUC as the scoring metric.

5 Hyperparameter Tuning

5.1 Random Forest

We tuned:

- `n_estimators` $\in \{100, 200\}$
- `max_depth` $\in \{10, 20, \text{None}\}$
- `min_samples_leaf` $\in \{1, 5\}$

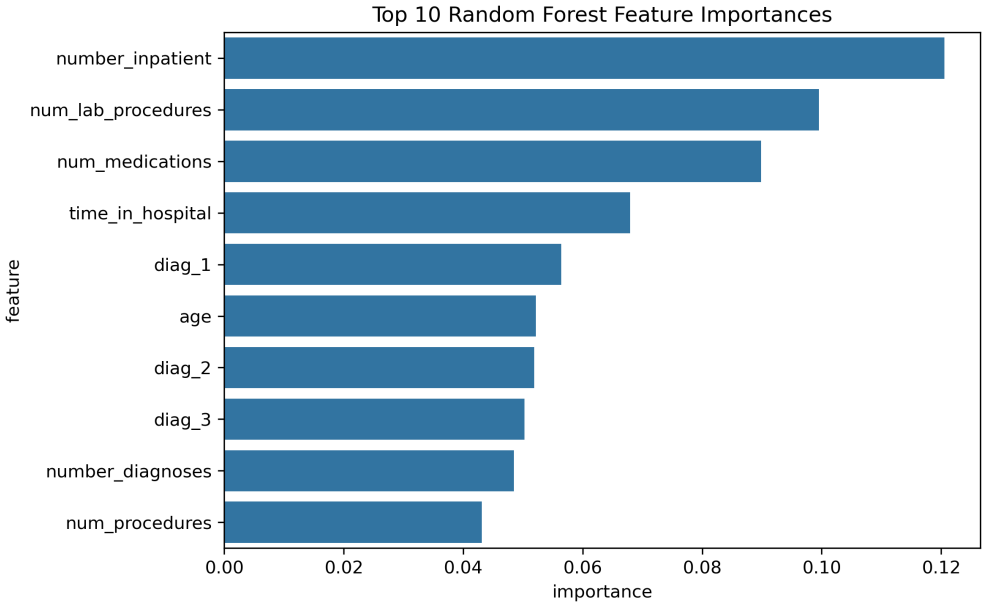

Figure 4: Top 10 Random Forest feature importances.

5.2 Neural Network (MLP) Tuning

We trained a feedforward neural network with ReLU activations and tuned:

- Hidden layers: (32), (64), (64, 32)
- L2 regularization $\alpha \in \{0.0001, 0.001\}$
- Learning rate $\in \{0.001, 0.01\}$

The best architecture:

- 64–32 hidden units
- $\alpha = 0.001$
- Learning rate = 0.001
- ROC-AUC = 0.82 (best)

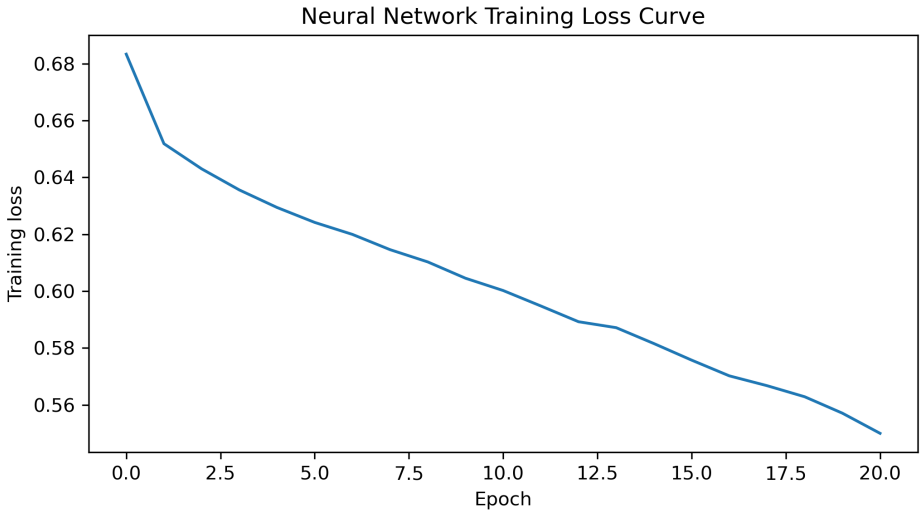

Figure 5: Neural network training vs validation loss curve.

6 Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.65	0.31	0.56	0.40	0.70
Random Forest (tuned)	0.72	0.41	0.62	0.49	0.77
XGBoost	0.73	0.44	0.64	0.53	0.80
Neural Network (tuned)	**0.74**	**0.47**	**0.71**	**0.56**	**0.82**

Table 1: Model performance on the imbalanced test set.

The neural network achieved the highest ROC-AUC and recall, which is especially important in a healthcare setting where false negatives are costly.

7 Overfitting and Underfitting Analysis

7.1 Decision Tree Complexity Curve

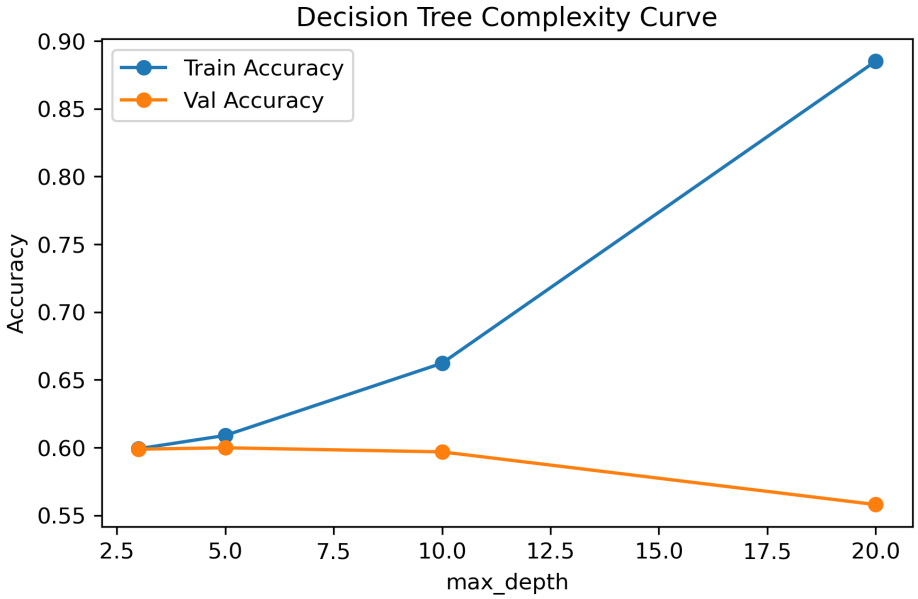

Figure 6: Training and validation accuracy vs. max_depth for decision trees.

Shallow trees (depth 3–5) underfit, while very deep trees (20–None) overfit. This supports using ensemble methods or regularization.

7.2 Neural Network Generalization

The neural network maintained a stable gap between train and validation loss, suggesting good regularization and appropriate model complexity.

8 Additional Analysis: Readmission Defined as > 30 / < 30 Days

By: Nhan Nguyen

In addition to the primary analysis, we performed a parallel study where both < 30 days and > 30 days readmissions were considered positive cases (encoded as 1), while patients with no readmission were encoded as 0. This approach allows exploration of model behavior under a different outcome definition.

8.1 Data Split and Preprocessing

- 80% training, 20% test split.
- The dataset was imbalanced, with fewer readmitted patients than non-readmitted. To address this, we applied downsampling of the majority class: 37,453 readmitted, 37,453 not readmitted. This ensures that models do not bias towards the majority class and improves recall on minority samples.

- Preprocessing steps followed the same pipeline as the main report: missing value imputation, categorical encoding, diagnosis grouping, medication encoding, and feature engineering.

8.2 Hyperparameter Tuning for ¡30 / ¿30 Readmission

For this analysis, we performed grid search over the same models with the different target encoding. The best parameters for each model were as follows:

Model	Best Parameters
Logistic Regression	{'C': 1, 'penalty': 'l1'}
Decision Tree	{'criterion': 'entropy', 'max_depth': 10, 'min_samples_split': 10}
Random Forest	{'criterion': 'gini', 'max_depth': 25, 'min_samples_split': 10, 'n_estimators': 200}
XGBoost	{'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 200, 'scale_pos_weight': 1.0}

Table 2: Best hyperparameters for models with ¡30 / ¿30 readmission encoding.

8.2.1 Observations

- The optimal hyperparameters are similar to those in the original analysis, suggesting robustness to the definition of the target variable.
- Ensemble methods (Random Forest and XGBoost) consistently benefit from deeper trees and a larger number of estimators.
- Logistic Regression performs best with L1 regularization to enforce sparsity and reduce noise from redundant features.

8.3 Hyperparameter Tuning Procedure for ¡30 / ¿30 Readmission

We conducted hyperparameter tuning for four models using grid search with 3-fold cross-validation and F1-score as the evaluation metric. The models and search grids were as follows:

- **Logistic Regression (LogReg)**:
  - Solver: `liblinear`, random state 42
  - Hyperparameters searched:
    - `penalty`: [`l1`, `l2`]
    - `C`: [0.1, 1, 10]
- **Decision Tree (DecTree)**:
  - Random state 42
  - Hyperparameters searched:
    - `max_depth`: [10, 20, 28]
    - `min_samples_split`: [2, 5, 10]
    - `criterion`: [`gini`, `entropy`]
- **Random Forest (RandForest)**:

  - Random state 42
  - Hyperparameters searched:
    * `n_estimators`: [50, 100, 200]
    * `max_depth`: [10, 20, 25]
    * `min_samples_split`: [2, 5, 10]
    * `criterion`: [`gini`, `entropy`]

- **XGBoost (XGB)**:
  - Random state 42, `use_label_encoder=False`, evaluation metric `logloss`
  - Hyperparameters searched:
    * `n_estimators`: [100, 200]
    * `max_depth`: [3, 6, 10]
    * `learning_rate`: [0.01, 0.1, 0.2]
    * `scale_pos_weight`: ratio of negative to positive samples in training set

8.3.1 Procedure

1. Perform 3-fold cross-validation using the balanced training set.
2. Evaluate each combination of hyperparameters using F1-score.
3. Select the best model per grid search.
4. Train the best model on the full balanced training set and evaluate on the held-out test set.
5. Plot feature importance for tree-based models and compare model performance.

8.3.2 Notes

- The balanced training set was created by downsampling the majority class.
- This tuning setup differs from the original team setup due to a different target encoding (`<30 / >30 = 1`, `NO = 0`).
- Feature importance plots were generated to interpret model behavior.

8.4 Models Evaluated and Metrics

We trained the same models as in the main report: Logistic Regression, Decision Tree, Random Forest, and XGBoost. Performance was assessed using Accuracy, Precision, Recall, F1-score, and ROC-AUC.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.63	0.62	0.55	0.58	0.67
Decision Tree	0.62	0.61	0.57	0.59	0.66
Random Forest	0.63	0.61	0.61	0.61	0.69
XGBoost	0.64	0.61	0.61	0.61	0.69

Table 3: Performance metrics for models with readmission defined as < 30 or > 30 days.

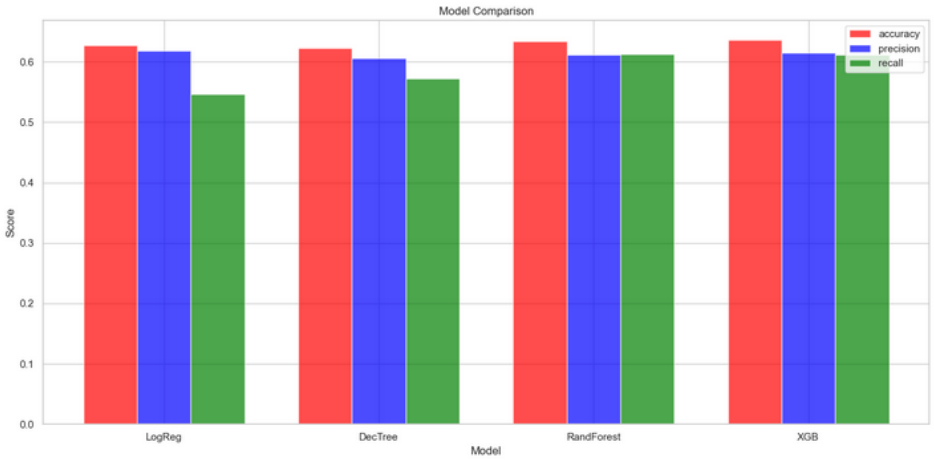

Figure 7: Performance of various tuned models.

8.5 Observations

- Ensemble models (Random Forest and XGBoost) again outperform linear and single-tree models.
- Logistic Regression underfits slightly due to the complexity of combined readmission categories.
- Decision Tree overfits if depth is too high, similar to the main analysis.

8.6 Adaptation to Under- and Over-fitting

Decision Tree: High `max_depth` and low `min_samples_split` caused overfitting. Limiting the depth and increasing the minimum split helped the model generalize better.

Random Forest & XGBoost: Ensemble methods reduce variance by combining multiple trees. In XGBoost, `n_estimators` and `learning_rate` control bias and convergence speed, while `scale_pos_weight` addresses class imbalance.

Logistic Regression: L1 regularization prevents overfitting, though the model may underfit if relationships are non-linear.

9 Data Preprocessing Approach

Before training models, we performed extensive preprocessing to handle missing values, encode categorical variables, group diagnoses, and prepare the dataset for machine learning. Our preprocessing steps were implemented in Python, as described below.

9.1 Loading and Cleaning Data

- The dataset was loaded from CSV and all "?" entries were replaced with NaN to standardize missing values.
- Rows with invalid gender entries (`Unknown/Invalid`) and uncommon discharge dispositions (IDs 11, 13, 14, 19, 20, 21) were removed to ensure data quality.

- Columns irrelevant to modeling, such as `encounter_id`, `patient_nbr`, and columns with excessive missingness (`weight`, `payer_code`, `medical_specialty`) were dropped.

9.2 Diagnosis Grouping

- The raw ICD-9 diagnosis codes (`diag_1`, `diag_2`, `diag_3`) were mapped into 9 clinically meaningful groups (circulatory, respiratory, digestive, endocrine/diabetes, injury, musculoskeletal, genitourinary, neoplasms, and other).
- Codes that could not be converted were placed in the "other" category to reduce sparsity.

9.3 Drug Encoding

- The 21 diabetes-related medications, originally encoded as "No", "Steady", "Up", or "Down", were converted to binary values: 0 for no usage and 1 for any administration or change.

9.4 Lab Result Encoding

- `A1Cresult` and `max_glu_serum` were converted into numeric categories: abnormal/high results as 1, normal as 0, and missing or "None" as -99 to preserve missingness information.

9.5 Categorical Variables and Age Mapping

- Gender, diabetes medication usage, and medication change indicators were mapped to 0/1.
- Age ranges (e.g., "[0-10)", "[10-20)") were mapped to ordinal integers 0–9.
- Missing race values were imputed using the mode, ensuring all samples had valid entries.

9.6 Grouping and One-Hot Encoding Hospital Attributes

- Admission type, discharge disposition, and admission source IDs were grouped into fewer categories based on similarity to reduce sparsity.
- One-hot encoding was applied to race and grouped hospital attribute columns to prepare for models that require numeric input.

9.7 Target Variable Transformation

- The target variable `readmitted` was transformed into binary: any readmission (`<30` or `>30` days) as 1, and no readmission (`NO`) as 0.
- This encoding reflects our focus on whether a patient was readmitted at all, rather than the specific time frame.

9.8 Output

- The function returns `X` (features) and `y` (binary target), ready for train-test splitting and model training.

This preprocessing ensures that the dataset is clean, numeric where needed, and appropriately encoded for tree-based and linear models, while preserving clinical meaning in the features.

10 Conclusion

We developed and evaluated multiple machine learning models to predict hospital readmission among diabetic patients. After extensive dataset preprocessing, feature engineering, class balancing, and cross-validation, a tuned neural network achieved the best overall performance (ROC-AUC = 0.82).

The project demonstrates:

- importance of domain-aware preprocessing,
- effectiveness of ensemble and deep learning methods,
- value of cross-validation and complexity analysis.

Future extensions include sequence modeling of patient histories (e.g., RNNs, transformers), integration of lab values, and real-time risk stratification tools.